Project 2 - Final Report

GitHub Link: https://github.com/albida33/ConvoSched

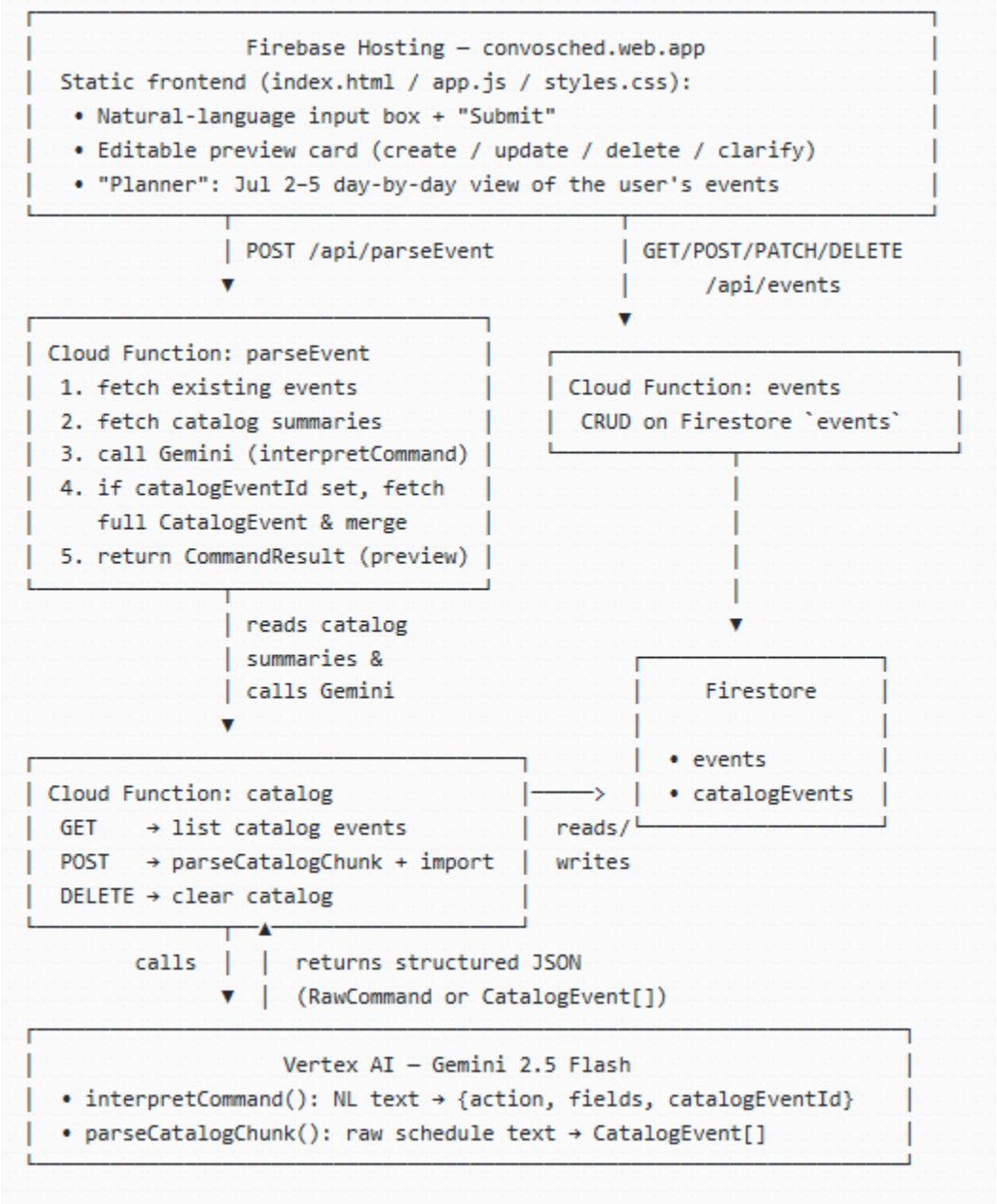
1. Research Process and Tools Explored

The project aimed to enable calendar management through natural language and automated event extraction from the Anime Expo 2026 schedule. The following technology stack was selected to achieve these goals:

- **Backend & Hosting (Firebase):** Utilizing Hosting, Cloud Functions, and Firestore for unified deployment. It provides seamless IAM integration with Vertex AI, allowing secure Gemini API calls without manual key management.
- **Natural Language Understanding (Vertex AI Gemini):** Employed Gemini with `responseSchema` for structured JSON output. This ensures model responses are treated as typed objects, simplifying frontend integration.
    - selected due to nativeness to gcloud system and cost-effectiveness compared to other Gemini models
- **Schedule Parsing (LLM-based):** Instead of traditional regex parsing, Gemini handles irregularities in the raw `ax-events.txt` file. This approach proved more robust for mapping 400+ entries into a structured `CatalogEvent[]` format.
- **Developer Tooling:** Used the Firebase CLI for deployment/debugging, Python for batch data analysis and imports, and TypeScript for a type-safe backend codebase.

2. Technical Design

a. High-Level System Architecture



b. Cloud Services Used

Service	Role
Firebase Hosting	Serves the static frontend at `convosched.web.app`; rewrites `/api/*` to the corresponding Cloud Function.
Cloud Functions for Firebase	Three HTTP functions —`parseEvent`, `events`, `catalog` — each `onRequest` with CORS enabled and a `us-central1` region (matching Firestore, to avoid cross-region latency).
Vertex AI — Gemini API	Called from within the Cloud Functions via the `@google-cloud/vertexai` SDK, using the function's default compute service account (granted `roles/aiplatform.user`) — no API key in any client or config file.
Cloud Firestore	Two collections:`events` (the user's personal calendar) and `catalogEvents` (437 pre-parsed Anime Expo 2026 panels).
Firebase CLI / Cloud Build / Artifact Registry	Used transparently by `firebase deploy` to build and push each function's container image.

c. Model(s) Used and Why

The project leverages **gemini-2.5-flash** for all AI operations, utilizing two specific prompt and schema configurations:

- **`interpretCommand`**: An OBJECT schema (RawCommand) defining actions (create/update/delete/clarify) and event metadata. It parses all incoming user requests.
- **`parseCatalogChunk`**: An ARRAY schema used solely for the initial catalog import, configured with high output token limits to batch-process 40+ events per call.

Selection Criteria for Gemini 2.5 Flash:

1. **Reliable Structured Output:** By enforcing `application/json` with a strict `responseSchema`, the model behaves like a typed function. This guarantees consistent output, allowing for direct integration into the UI preview and Firestore without risky parsing.
2. **Operational Efficiency:** The "flash" tier provides the optimal balance of low latency and cost-effectiveness for single-step reasoning tasks like intent classification and schedule formatting.

3. **Extensive Context Window:** The model easily processes the full catalog summary (~25K tokens) alongside the user's existing events, ensuring comprehensive matching without data loss.

d. Data Flow and User Interaction Flow

The system uses a unified processing pipeline for both personal events and convention catalog items:

1. **User Request:** The user submits a natural language request via the frontend.
2. **Backend Parsing:** The frontend sends the request to `/api/parseEvent`. The backend loads the user's existing events and a compact catalog summary, then calls the `interpretCommand` function (Gemini).
3. **AI Reasoning:** Gemini processes the input:
    a. **Personal Events:** Parses intent (create/update/delete). For updates, it identifies the `eventId` and returns only the changed fields.
    b. **Catalog Matching:** If the request references a convention panel, Gemini returns a "create" action with a `catalogEventId`. The system then fetches the full catalog document (including description) from Firestore to ensure rich data availability.
4. **Unified Preview:** The backend merges the AI-generated command with existing data (null-coalescing for updates). It returns a `CommandResult` containing a preview. The user views this in an editable card.
5. **Confirmation:** The user confirms the change. **No AI output is written to the database without this explicit verification.**
6. **Persistence:** Upon confirmation, the frontend writes to Firestore (POST/PATCH/DELETE via `/api/events`).
7. **Refresh:** The frontend reloads the data and re-renders the **Planner**, grouping events by date.

---

3. Evaluation

a. What Tasks Does It Help With?

- **Creating a personal event, updating/Deleting an existing event (personal or convention panel)** by referring to it using natural language
- **Disambiguating** when a request could refer to multiple convention sessions (e.g., multiple "Maid Cafe" time slots).
- **Viewing a day-by-day itinerary** ("Planner") spanning the four days of Anime Expo 2026, separate from other personal events.

b. How Accurate or Useful Are the Responses?

The responses are generally correct. The structured-output approach meant that, once a response came back as valid JSON, it was *usable* essentially 100% of the time — there were no cases of malformed JSON breaking the frontend. The remaining accuracy issues were at the **decision** level (did Gemini pick the right action/match?), not at the **format** level.

c. Limitations or Failures Observed

- **Inconsistent Matching:** Gemini sometimes fails to identify specific catalog entries, likely due to the large volume of data (437 entries) in the prompt.
- **Source Duplicates:** The parser includes redundant entries found in the original source schedule.
- **Scalability:** Passing the entire 10–25K token catalog for every request creates unnecessary overhead, leading to slower response times.
- **Access Control:** The prototype lacks authentication, making it unsuitable for multi-user environments.

d. How Does It Compare to Not Using an AI Assistant?

Without ConvoSched, adding panels from a 437-entry convention schedule to a personal calendar means: searching the published schedule (a dense PDF/website) for something of interest, manually noting its date, time, and room, then switching to a calendar app and re-typing all of that — repeated for every panel of interest, across four days. This is slow and error-prone (easy to mistype a time or miss that two same-named sessions occur on different days).

With ConvoSched, the same task is a single sentence: the correct panel (disambiguated if necessary) is located, and its title, time, room, and description are populated automatically into a reviewable preview.

---

4. Challenges Faced and How They Were Solved

1. Designing token-efficient catalog matching. Sending all 437 catalog events *with descriptions* on every request would be prohibitively large.

Solution: split the catalog representation into a **compact summary** (id, title, time range, location — no description) sent to Gemini for matching, and a **full record** (with description) fetched from Firestore *only when there's an actual match* (a single cheap document read), then merged into the preview server-side.

2. Duplicate catalog entries from retried requests. After the above fix, the catalog had 474 documents (expected 437) — analysis showed 140 duplicate documents, consistent with several chunks having been processed twice during the earlier 502-affected run.

Solution: cleared the entire `catalogEvents` collection (`DELETE /api/catalog`) and re-ran the full import via the direct Cloud Run URL with sequential requests and short delays between chunks. The result was exactly 437 documents, with each chunk's reported count matching its input block count precisely.

3. Two chunks failing with a clean error during re-import. During the clean re-import, two chunks (out of eleven) returned a proper JSON error (`"Failed to parse/import catalog chunk"`) rather than 502s — likely a transient Vertex AI rate-limit immediately following a prior large call.

Solution: re-ran just those two chunks individually with a longer delay (15s) between requests; both succeeded on retry, bringing the total to 437/437.

---

5. What I Learned About Cloud AI Systems

- **Managing Non-Determinism:** LLM outputs can vary even for simple tasks, so backend systems must be defensive—treating model output as partial data and normalizing values rather than assuming complete consistency.
- **Human-in-the-Loop Safety:** The "interpret → preview → confirm → write" pattern creates a critical safety net, ensuring no data is saved to the database without explicit user verification.
- **Proactive Scaling:** Designing for constraints (like token limits) early—such as using compact data summaries for AI while fetching full details from the database—prevents costly redesigns as data volume increases.
- **Reusable Data Pipelines:** Building one-time tasks (like data importers) as persistent application code allows them to leverage existing production infrastructure, making them easier to maintain, secure, and reuse.

---

6. Future Improvements

1. **Add authentication and per-user data.** Firebase Authentication plus Firestore security rules keyed on the already-reserved `userId` field would turn this from a single-shared-list prototype into a real multi-user app.
2. **Replace "send the whole catalog every time" with retrieval.** As the catalog grows (more conventions, multiple years), pre-filtering candidate catalog entries — via keyword

search or vector embeddings — before calling Gemini would keep prompt size and cost roughly constant regardless of catalog size, and likely also improve match accuracy by reducing the needle-in-haystack problem.

3. **Generalize the Planner.** The four-day range (July 2–5, 2026) is currently hardcoded; a future version should derive the date range from the active catalog's date span (or let the user configure it), and could grow into a full calendar-grid view.
4. **Support multi-event requests** ("Add the Welcome Ceremony and the Maid Cafe on July 3 to my calendar") by allowing `interpretCommand` to return multiple commands per request.